PATRONES DE DISEÑO · CLASE 01

# El patrón Observer

Notificación desacoplada de cambios de estado — con ejemplos en C# y Visual Studio

Universidad ICES · Nivel: desarrolladores · .NET / C#

```
subject.PropertyChanged += (s, e) =>
    Console.WriteLine($"Cambió {e.PropertyName}");
```

RECORRIDO

# Agenda

- Qué problema resuelve Observer y cuál es su intención
- Participantes y estructura del patrón
- Tres formas de implementarlo en C#: interfaces, events e IObservable<T>
- Beneficios reales y costos ocultos
- Dónde conviene usarlo y dónde claramente no
- Errores frecuentes: fugas de memoria, orden y excepciones
- Ejercicio integrador para resolver

MOTIVACIÓN

# El problema que resuelve

Un objeto cambia de estado y varios otros necesitan enterarse. Si el que cambia conoce a cada interesado, cada nuevo interesado obliga a recompilar y modificar la clase original.

### Acoplamiento directo

El emisor instancia y llama a cada receptor concreto.

Agregar un destino = tocar el emisor.

Viola Open/Closed y dificulta las pruebas unitarias.

### Con Observer

El emisor solo conoce una abstracción de suscriptor.

Los interesados se agregan y quitan en tiempo de ejecución.

El emisor no cambia nunca al sumar observadores.

DEFINICIÓN

# Intención y participantes

**Intención (GoF):** definir una dependencia uno-a-muchos entre objetos, de modo que cuando uno cambia de estado, todos sus dependientes son notificados y actualizados automáticamente.

| | |
|---|---|
| **Subject (Sujeto)** | Mantiene la lista de observadores y expone Attach / Detach. |
| **Observer** | Interfaz con el método de actualización que invoca el sujeto. |
| **ConcreteSubject** | Guarda el estado de interés y dispara Notify al cambiar. |
| **ConcreteObserver** | Reacciona al aviso y consulta o recibe el nuevo estado. |

MECÁNICA

# Flujo de notificación: push y pull

**1**

### Suscribir

El observador se registra en el sujeto.

**2**

### Cambio

El estado del sujeto se modifica.

**3**

### Notificar

El sujeto recorre la lista y avisa.

**4**

### Reaccionar

Cada observador ejecuta su lógica.

**Push** el sujeto envía los datos ya listos en la notificación. Simple y eficiente cuando todos los observadores necesitan lo mismo.

**Pull** el sujeto avisa que algo cambió y cada observador consulta lo que le interesa. Más flexible, pero acopla al observador con la API del sujeto.

```
// push: OnStockChanged(new StockArgs(symbol, price));
// pull: OnChanged();  →  var p = subject.Price;
```

# Implementación clásica con interfaces

```
public interface IObserverDeStock {
    void Actualizar(string simbolo, decimal precio);
}

public class Cotizacion {
    private readonly List<IObserverDeStock> _obs = new();
    private decimal _precio;

    public void Suscribir(IObserverDeStock o) => _obs.Add(o);
    public void Desuscribir(IObserverDeStock o) => _obs.Remove(o);

    public decimal Precio {
        get => _precio;
        set { if (_precio == value) return; _precio = value; Notificar(); }
    }

    private void Notificar() {
        foreach (var o in _obs.ToArray())   // copia: evita mutación
            o.Actualizar("MSFT", _precio);
    }
}
```

C# · RECOMENDADO

# La forma idiomática: eventos y delegados

En .NET el patrón ya está en el lenguaje. Un event es un Subject con Attach (+=) y Detach (-=) incorporados.

```
public class PrecioArgs : EventArgs {
    public decimal Precio { get; init; }
}

public class Cotizacion {
    public event EventHandler<PrecioArgs>? PrecioCambiado;

    private decimal _precio;
    public decimal Precio {
        get => _precio;
        set {
            if (_precio == value) return;
            _precio = value;
            PrecioCambiado?.Invoke(this, new PrecioArgs { Precio = value });
        }
    }
}

void Handler(object? s, PrecioArgs e) => Console.WriteLine(e.Precio);
c.PrecioCambiado += Handler;   // suscripción
c.PrecioCambiado -= Handler;   // baja: imprescindible
```

# IObservable<T>, INotifyPropertyChanged y Rx

### INotifyPropertyChanged

El Observer que usa el binding de WPF, MAUI y WinForms.

### IObservable<T> / IObserver<T>

Contrato estándar de .NET con OnNext, OnError y OnCompleted.

### System.Reactive (Rx)

Composición de flujos: filtrar, transformar y limitar frecuencia.

```
public class Sensor : IObservable<double> { /* ... */ }

// Con Rx: solo temperaturas altas, como máximo una por segundo
sensor.Where(t => t > 80)
      .Throttle(TimeSpan.FromSeconds(1))
      .Subscribe(t => _alertas.Enviar($"Temperatura {t} °C"));

// Subscribe devuelve IDisposable: dispose == desuscribirse
using var suscripcion = sensor.Subscribe(observador);
```

POR QUÉ USARLO

# Beneficios

### Bajo acoplamiento

El sujeto solo conoce una abstracción; ignora quién escucha y cuántos son.

### Abierto/Cerrado

Sumás observadores nuevos sin tocar ni recompilar el sujeto.

### Relaciones dinámicas

Alta y baja de suscriptores en tiempo de ejecución.

### Difusión uno-a-muchos

Un solo cambio alimenta UI, log, caché y auditoría a la vez.

### Testeabilidad

Se verifica con un observador falso que registra las notificaciones.

### Base de otros modelos

Sustenta MVVM, event-driven, colas de mensajes y programación reactiva.

# Dónde usarlo

- Interfaces de usuario: el modelo cambia y la vista se actualiza sola (MVVM, data binding).
- Eventos de dominio: al confirmar un pedido se dispara facturación, stock y correo al cliente.
- Telemetría y sensores: un flujo continuo alimenta gráfico, alertas y almacenamiento.
- Logging y auditoría transversales, sin ensuciar la lógica de negocio.
- Caché e invalidación: al cambiar el dato se avisa a quienes lo tienen guardado.
- Sistemas de plugins: los módulos se enganchan a puntos de extensión sin conocerse entre sí.
- Cuando la cantidad de interesados es desconocida o variable en tiempo de ejecución.

# Dónde no usarlo

- Hay un único consumidor fijo y conocido: una llamada directa es más clara y más rápida.
- Necesitás garantías de orden: el orden de notificación no está definido por el patrón.
- El flujo requiere transacciones o respuesta del receptor; Observer es aviso, no petición-respuesta.
- Cadenas en cascada donde un observador modifica el sujeto: reentrada, bucles y difícil depuración.
- Rutas críticas de rendimiento: el despacho a muchos suscriptores agrega costo por evento.
- Comunicación entre procesos o servicios: ahí corresponde un bus de mensajes, no eventos en memoria.
- Si el flujo de control ya es difícil de seguir, sumar indirección lo empeora.

CUIDADO CON ESTO

# Trampas frecuentes en .NET

### Fugas de memoria por suscripción

El sujeto mantiene viva la referencia al observador. Si nunca hacés -= o Dispose, el objeto no se libera. Es la causa número uno de leaks en WPF.

### Excepciones en un handler

Una excepción en un suscriptor corta la iteración y los siguientes nunca se enteran. Aislá cada llamada con try/catch.

### Modificar la lista al notificar

Si un observador se desuscribe durante Notify, la colección cambia mientras se recorre. Iterá sobre una copia.

### Hilos y concurrencia

Los handlers corren en el hilo que dispara el evento. Para tocar la UI hay que volver al hilo de interfaz (Dispatcher).

CHECKLIST

# Buenas prácticas

- Preferí event EventHandler<T> antes que una lista de interfaces hecha a mano.
- Invocá siempre con el operador condicional: PrecioCambiado?.Invoke(this, args).
- Devolvé un IDisposable en Subscribe para que dar de baja sea imposible de olvidar.
- Desuscribí en Dispose, Unloaded o el cierre de la ventana; considerá referencias débiles.
- Mantené los handlers cortos y sin efectos que vuelvan a modificar el sujeto.
- Definí bien la carga útil del evento: qué cambió y cuál es el nuevo valor.
- En Visual Studio, poné un punto de interrupción dentro del handler y mirá la Pila de llamadas para seguir el disparo.

PARA RESOLVER

# Ejercicio integrador

**Consigna** Aplicación de consola en Visual Studio (.NET 8). Modelar una estación meteorológica que publica lecturas de temperatura y humedad a varios paneles independientes.

- EstacionMeteorologica expone un evento con temperatura, humedad y marca de tiempo.
- Implementá tres observadores: PanelActual, PanelEstadisticas (mín, máx, promedio) y PanelAlertas (avisa sobre 35 °C).
- El registro y la baja de paneles deben poder hacerse en tiempo de ejecución desde un menú de consola.
- Que una excepción en un panel no impida que los demás reciban la lectura.
- Subscribe debe devolver un IDisposable que dé de baja al panel al liberarse.

```
// Punto de partida
public record Lectura(double TempC, double Humedad, DateTime Momento);
public interface IPanel { void Recibir(Lectura l); }
public class EstacionMeteorologica {
    public IDisposable Suscribir(IPanel panel) => /* TODO */;
    public void Publicar(Lectura l) => /* TODO: notificar a todos */;
}
```

EJERCICIO · PARTE 2

# Entrega y criterios de evaluación

### Extensiones (punto extra)

Reescribí la solución usando event EventHandler<LecturaArgs>.

Agregá una versión con IObservable<Lectura> y compará ambas.

### Entrega

Solución de Visual Studio comprimida o enlace a repositorio.

Un README breve con la decisión de diseño; debe compilar sin ajustes.

| | |
|---|---|
| Correcta aplicación del patrón | **35 %** |
| Manejo de suscripción y baja sin fugas | **25 %** |
| Aislamiento de errores en los observadores | **20 %** |
| Claridad, nombres y pruebas | **20 %** |

# Para llevarse

## Observer no es un evento: es la decisión de que quien produce el cambio no conozca a quien reacciona.

- En C# empezá siempre por event; recurrí a interfaces propias o a Rx solo cuando el caso lo justifique.
- El costo del patrón es la trazabilidad: se gana flexibilidad y se pierde flujo lineal.
- Toda suscripción necesita un plan de baja desde el día uno.
- Lecturas: Design Patterns (GoF, cap. Observer) y la documentación de eventos de .NET.